BÁO CÁO THỰC HÀNH

Thực hành An toàn mạng máy tính

Lab X: NỘI DUNG XYZ

GVTH: Nguyễn Ngọc Trưởng

Ngày báo cáo: 21/10/2026

Nhóm X – NT101.QXX.X

THÔNG TIN CHUNG

MSSV	Họ và tên	Nội dung báo cáo	% công việc
2451664	Phạm Thanh Thiên	Câu 2.2	50%
24521708	Trương Đức Thịnh	Câu 2.3	50%

LINK GIT

Nhiệm vụ 2.2

2.2.0. Môi trường thực hiện

- **Hệ điều hành:** Ubuntu 22.04 LTS.
- **Công cụ sử dụng:** xxd, cmp, md5sum, sha1sum (thuộc gói coreutils/xxd có sẵn trê Ubuntu 22.04), trình xem PDF của hệ thống.
- **Hỗ trợ chạy tệp thực thi 32-bit:** hai chương trình hello và erase là tệp 32-bit nên cần bật kiến trúc i386 và cài thư viện libc6:i386 theo hướng dẫn của đề bài:

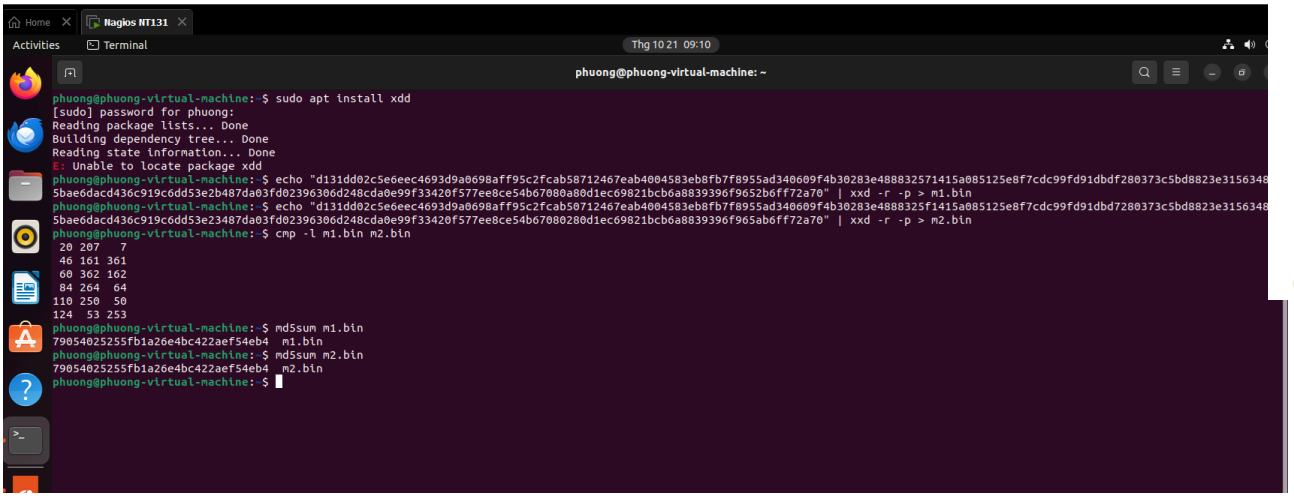
2.2.1. Va chạm MD5 trên hai thông điệp dạng HEX

Phương pháp thực hiện. Chuỗi HEX của Message 1 và Message 2 được lưu vào hai tệp văn bản (msg1.hex, msg2.hex), sau đó chuyển về dạng nhị phân bằng xxd. Tiếp theo, dùng cmp để đếm số byte khác nhau và dùng md5sum để tính giá trị băm MD5 của từng thông điệp.

```
xxd -r -p msg1.hex msg1.bin
xxd -r -p msg2.hex msg2.bin
md5sum msg1.bin
md5sum msg2.bin
```

Kết quả quan sát.

- Mỗi thông điệp có độ dài 128 byte (256 ký tự HEX). Hai thông điệp khác nhau đúng 6 byte, tại các vị trí (đánh số từ 0) 19, 45, 59, 83, 109 và 123. Lệnh cmp -l đánh số từ 1 nên sẽ báo các vị trí tương ứng là 20, 46, 60, 84, 110 và 124
- Mặc dù nội dung khác nhau, hai thông điệp cho cùng một giá trị MD5
- Như vậy MD5 không còn đảm bảo tính kháng va chạm: tồn tại hai đầu vào khác nhau có cùng giá trị băm



Hình 1. Kết quả so sánh và băm MD5 của hai thông điệp

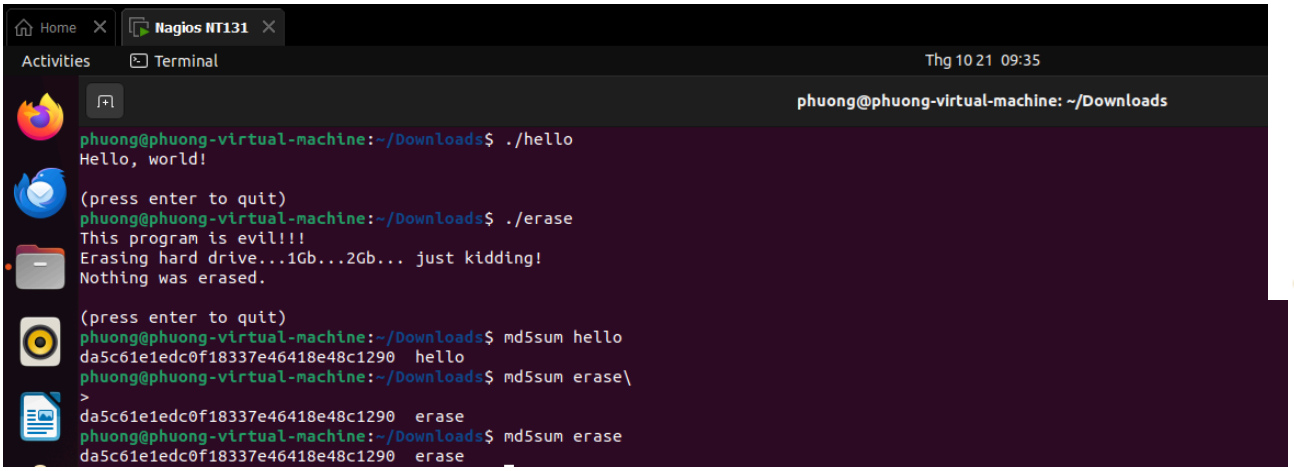
2.2.2. Va chạm MD5 trên tệp thực thi

Phương pháp thực hiện. Cấp quyền thực thi cho hai chương trình hello và erase, chạy từng chương trình từ console để quan sát hành vi, sau đó tính giá trị MD5 của chúng

```
chmod +x hello erase
./hello
./erase
md5sum hello
md5sum erase
```

Kết quả quan sát.

- Hai chương trình có hành vi khác nhau khi chạy: hello in ra thông điệp chào, còn erase thực thi một chức năng khác
- Hai tệp nhị phân không giống nhau (cmp báo có byte khác biệt) nhưng lại có cùng giá trị MD5
- Ý nghĩa: kẻ tấn công có thể tạo một chương trình vô hại và một chương trình độc hại có cùng MD5. Nếu hệ thống chỉ dựa vào MD5 để kiểm tra tính toàn vẹn hoặc danh sách cho phép, chương trình độc hại sẽ bị nhận nhầm là chương trình hợp lệ



Hình 2. Hành vi của hai chương trình và giá trị MD5 của chúng

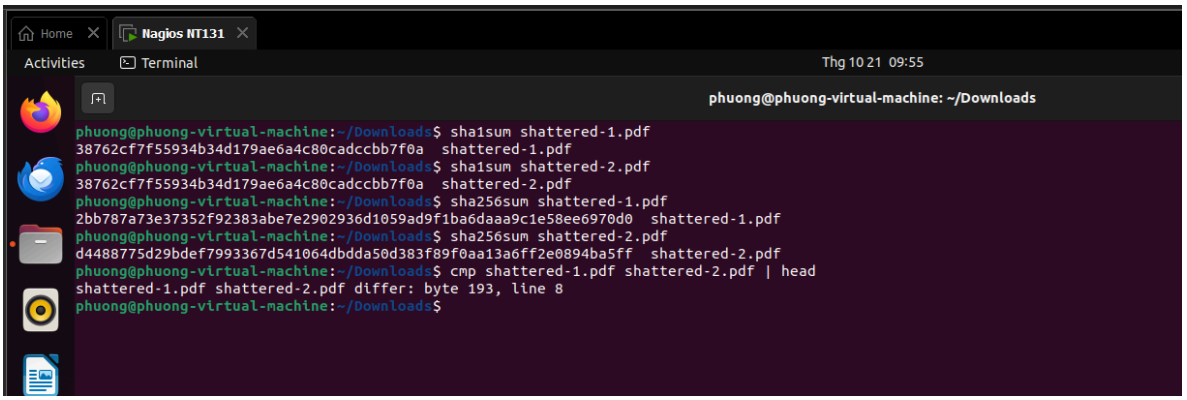
2.2.3. Va chạm SHA-1 trên tệp PDF

Phương pháp thực hiện. Tải hai tệp shattered-1.pdf và shattered-2.pdf, mở bằng trình xem PDF để so sánh nội dung hiển thị, sau đó tính SHA-1 bằng sha1sum. Để đối chứng, chạy thêm sha256sum để thấy hai tệp vẫn phân biệt được bằng hàm băm mạnh hơn

```
sha1sum shattered-1.pdf shattered-2.pdf
sha256sum shattered-1.pdf shattered-2.pdf
cmp shattered-1.pdf shattered-2.pdf | head
```

Kết quả quan sát.

- Về trực quan, hai tệp PDF hiển thị khác nhau: nội dung giống nhau nhưng màu nền của phần văn bản khác nhau (một tệp nền đỏ, tệp còn lại nền xanh).
- Hai tệp có cùng giá trị SHA-1: 38762cf7f55934b34d179ae6a4c80cadccbb7f0a.
- Giá trị SHA-256 của hai tệp khác nhau, chứng tỏ hai tệp thực sự là hai tệp khác nhau và chỉ SHA-1 bị va chạm



Hình 3. Hai tệp PDF và giá trị băm SHA-1, SHA-256

2.2.4. Kết luận

Từ ba thí nghiệm trên có thể kết luận: MD5 (đầu ra 128 bit) và SHA-1 (đầu ra 160 bit) đã b phá vỡ tính kháng va chạm (collision resistance), nghĩa là có thể tìm được hai đầu vào khá nhau M1 ≠ M2 sao cho H(M1) = H(M2). Hai hàm này không còn an toàn cho các mục đích cần tính toàn vẹn và chống giả mạo như chữ ký số hay chứng chỉ số.

Lý do tồn tại xung đột:

- **Nguyên lý chuồng bồ câu:** không gian đầu vào là vô hạn nhưng không gian đầu ra hữu hạn (2^128 với MD5, 2^160 với SHA-1), nên va chạm luôn tồn tại về mặt lý thuyết. Một hàm băm tốt chỉ cần làm cho việc tìm ra chúng khó về mặt tính toán.
- **Giới hạn tấn công ngày sinh (birthday attack):** với đầu ra n bit, chi phí tìm va chạm lý tưởng khoảng 2^(n/2), tức khoảng 2^64 với MD5 và 2^80 với SHA-1.
- **Điểm yếu trong thiết kế:** các nhà nghiên cứu (Wang và cộng sự với MD5; nhóm CWI và Google với SHA-1, tấn công SHAttered) đã phân tích vi sai trên các vòng nén, tạo ra các khối thông điệp gây sai lệch có kiểm soát mà bị triệt tiêu ở các bước sau. Nhờ đó chi phí tìm va chạm thực tế thấp hơn nhiều so với giới hạn ngày sinh. Với MD5, va chạm có thể tạo trong vài giây trên máy tính thông thường.
- **Cấu trúc Merkle-Damgard:** nếu H(M1) = H(M2) và hai thông điệp có cùng độ dài thì H(M1 ∥ S) = H(M2 ∥ S) với mọi hậu tố S. Đây là cơ sở để chèn cùng một phần nội dung phía sau (như phần còn lại của PDF hoặc mã chương trình trong hello/erase) mà hai giá trị băm vẫn trùng nhau. Nhờ vậy kẻ tấn công tạo được hai tệp có ý nghĩa hoàn toàn khác nhau (một tệp lành tính, một tệp độc hại) nhưng cùng giá trị băm.

Khuyến nghị: không dùng MD5 và SHA-1 cho chữ ký số, chứng chỉ hay kiểm tra toàn vẹn quan trọng; thay bằng họ SHA-2 (SHA-256, SHA-512) hoặc SHA-3

Nhiệm vụ 2.3

Chuẩn bị

- Môi trường: Colab
- Cài đặt fastcoil: https://github.com/zhijieshi/md5collgen.git

```
/content/md5collgen# ./md5collgen
MD5 collision generator v1.5
by Marc Stevens (http://www.win.tue.nl/hashclash/)


Allowed options:
  -h [ --help ]           Show options.
  -q [ --quiet ]          Be less verbose.
  -i [ --ihv ] init_val   Use specified initial value. Default is MD
5 initial
                          value.
  -p [ --prefixfile ] fn  Calculate initial value using given prefix
file. Also
                          copies data to output files.
  -o [ --out ] fn1 fn2    Set output filenames. This must be the las
t option
                          and exactly 2 filenames must be specified.
                          Default: -o msg1.bin msg2.bin
```

+ Cài đặt thành công

Mục TIêu:

- Tạo hai tệp có nội dung khác nhau nhưng có cùng giá trị băm md5 từ 1 prefix cho trước bằng công cụ md5collgen. Qua đó quan sát cách công cụ hoạt động hiểu vì sao MD5 không còn an toàn nữa

Cơ sở lý thuyết

- Cấu trúc MD5, Va chạm, Indentical_prefix collision

Thực hiện:

Câu 1 Nhiệm vụ 2.3

- Tạo prefix và sinh collision

```
/content/lab03_task23# echo -n "This is the prefix for lab 03 - UIT"
 > prefix.txt
/content/lab03_task23# wc -c prefix.txt
35 prefix.txt
```

```
/content/lab03_task23# md5collgen -p prefix.txt -o out1.bin out2.bin
MD5 collision generator v1.5
by Marc Stevens (http://www.win.tue.nl/hashclash/)

Using output filenames: 'out1.bin' and 'out2.bin'
Using prefixfile: 'prefix.txt'
Using initial value: c46c7d12fb776b717764389061f29cf7

Generating first block: ..........
Generating second block: S01.........
Running time: 2.673083 s
/content/lab03_task23# wc -c out1.bin out2.bin
192 out1.bin
192 out2.bin
384 total
/content/lab03_task23# md5sum out1.bin out2.bin
6cad851ff21321f556b517c167cdfc48  out1.bin
6cad851ff21321f556b517c167cdfc48  out2.bin
```

```
/content/lab03_task23# sha256sum out1.bin out2.bin
6193e0e00bf8a903b432e1aa2247ffe7f53b59e18e2340b1f40348dbe88bdb94  ou
t1.bin
19c182645eb6c82e48c573382a7a4c7475edc094848296d6eda7f85966b9a762  ou
t2.bin
```

```
/content/lab03_task23# cmp -l out1.bin out2.bin | wc -l
7
```

```
/content/lab03_task23# diff out1.bin out2.bin
Binary files out1.bin and out2.bin differ
/content/lab03_task23#
/content/lab03_task23# xxd out1.bin
00000000: 5468 6973 2069 7320 7468 6520 7072 6566  This is the pref
00000010: 6978 2066 6f72 206c 6162 2030 3320 2d20  ix for lab 03 -
00000020: 5549 5400 0000 0000 0000 0000 0000 0000  UIT.............
00000030: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000040: 0059 3185 f1ab 943c 3ddb 5936 4458 f44e  .Y1....<=.Y6DX.N
00000050: db78 4fbf 4eb9 f451 2e22 3966 3130 8adf  .xO.N..Q."9f10..
00000060: 88d9 add0 8144 f29d 4fc8 e11a 2176 1f8a  .....D..O...!v..
00000070: 8201 6cfa 1f20 75b1 48d1 265b b17e b6c4  ..l.. u.H.&[.~..
00000080: db8e fc8c 875c 6fa6 a31e 115d 33fc ce12  .....\o....]3...
00000090: dedd 1972 2366 465e d35a f6b9 ec46 de69  ...r#fF^.Z...F.i
000000a0: 3065 265b f6d6 df7d ba92 4db7 9f28 4b2c  0e&[...}..M..(K,
000000b0: f92f c6d7 1db0 1530 b8b1 5048 c601 f42f  ./.....0..PH.../
```

Nhận xét:

- Hai tệp có MD5 giống nhau nhưng SHA-256 khác nhau, chứng tỏ nội dung thật sự khác nhau, chỉ MD5 bị va chạm. Prefix dài [..] byte, không chia hết cho 64, nên md5collgen tự động đệm các byte 0 cho đủ bội số của 64 (đến [..] byte). Sau đó công cụ nối thêm 128 byte (hai khối 64 byte) chứa dữ liệu collision, phần này khác nhau giữa hai tệp. Do đó kích thước đầu ra bằng ⌈độ dài prefix / 64⌉ × 64 + 128 = [..] byte, khớp với quan sát. Hai tệp chỉ khác nhau ở [..] byte, đều nằm trong 128 byte cuối.

Trả lời câu hỏi:

- Khi độ dài prefix không là bội số của 64, công cụ tự đệm byte 0 vào sau prefix cho đủ bội số của 64 rồi mới nối khối collision. Việc căn theo khối là bắt buộc vì MD5 xử lý từng khối 64 byte và collision chỉ được tạo trên ranh giới khối.

Câu 2_Nhiệm vụ 2.3

```
head -c 64 /dev/zero | tr '\0' 'A' > prefix64.txt
md5collgen -p prefix64.txt -o out64_1.bin out64_2.bin
```

```
/content/lab03_task23# wc -c out64_1.bin out64_2.bin
192 out64_1.bin
192 out64_2.bin
384 total
```

```
/content/lab03_task23# md5sum out64_1.bin out64_2.bin
99840eaca00e04fd6e450e2b308ab9bf  out64_1.bin
99840eaca00e04fd6e450e2b308ab9bf  out64_2.bin
/content/lab03_task23# cmp -l out64_1.bin out64_2.bin | wc -l
6
```



```
/content/lab03_task23# xxd out64_1.bin
00000000: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
00000010: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
00000020: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
00000030: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
00000040: 8b9b 8532 e286 de44 87d4 00e1 7e26 d27d  ...2...D....~&.}
00000050: 4731 b76d 4301 4b6f 368a 815d 6fac 3237  G1.mC.Ko6..]o.27
00000060: 24f1 07e9 6ca4 e9ae 07be d521 f573 d892  $...l......!.s..
00000070: 2811 38c7 f277 4ec9 3621 9b9e 601e e864  (.8..wN.6!..`..d
00000080: 471f 12de 3cd7 634c dbc8 be6a d971 a6b5  G...<.cL...j.q..
00000090: 5672 17a5 03a3 bced f098 d8b7 cf0b 6ef0  Vr............n.
000000a0: d157 bedf 1654 bd46 0632 11aa 8098 c63c  .W...T.F.2.....<
000000b0: ea7c 6001 76d7 03d7 56d8 30e3 865f f8c6  .|`.v...V.0._..
```

Nhận xét:

- Prefix đã là bội số của 64 nên không có padding. 64 byte đầu của hai tệp chính là prefix, 128 byte còn lại là phần collision, tổng cộng 192 byte. MD5 hai tệp vẫn giống nhau.

So sánh hai trường hợp:

	Prefix không bội 64	**Prefix 64 byte**
Padding	**Có**	Không
Kích thước đầu ra	**[...]**	[192]
MD5 hai tệp	**Giống nhau**	**Giống nhau**

Kết luận

- Với md5collgen, ta tạo được hai tệp khác nhau cùng MD5 chỉ trong thời gian rất ngắn. Công cụ luôn căn prefix theo bội số của 64 byte (đệm 0 nếu cần) rồi nối 128 byte collision. Kết hợp với tính chất mở rộng, kẻ tấn công có thể tạo cặp tệp có phần nội dung khác nhau nhưng cùng MD5, như hai chương trình hello và erase ở Nhiệm vụ 2.2. Vì vậy MD5 không còn an toàn cho chữ ký số hay kiểm tra toàn vẹn, nên dùng SHA-256 trở lên.

Trả lời câu hỏi:

- Khi độ dài prefix không là bội số của 64, công cụ tự đệm byte 0 vào sau prefix cho đủ bội số của 64 rồi mới nối khối collision. Việc căn theo khối là bắt buộc vì MD5 xử lý từng khối 64 byte và collision chỉ được tạo trên ranh giới khối.

HẾT